

KAM Intelligence Platform

Code Understanding and POC Preparation Guide

Prepared for presentation readiness - generated June 7, 2026

Use this document as your map for explaining what the project does, how the Next.js and TypeScript code is organized, where the AI models are called, how scoring works, and what the new POC screen demonstrates.

Presentation angle	Short answer
Product story	A Key Account Management intelligence platform that turns account data, documents, signals, KYC, and scoring into actionable portfolio decisions.
Technical story	A Next.js App Router application with React client screens, server API route handlers, Prisma/MySQL persistence, AI provider abstraction, mock integration adapters, scoring logic, and agent-style AI workflows.
POC story	A direct working screen where a document is uploaded, parsed, extracted into account fields/KYC/scoring, corrected through chat, and recalculated using the lead framework.

1. The One-Minute Product Explanation

KAM Intelligence helps account managers understand account health without manually stitching together spreadsheets, meeting notes, contracts, support signals, financial data, and relationship history. The platform gives each account a health score, highlights risks and opportunities, drafts KYC intelligence, proposes actions, and supports portfolio-level visibility.

For the presentation, keep the story simple: the system ingests account context, uses deterministic scoring plus AI interpretation, shows explainable score drivers, and lets humans accept, reject, correct, or approve the outputs.

The main user roles

- **Associate:** supports KAMs, uploads documents, drafts account setup/KYC, and routes changes for review.
- **KAM:** owns the account, reviews proposals, manages account profile/actions/documents, and approves changes.
- **C-Level/Executive:** read-only portfolio visibility for health, risk, and strategic decisions.

What changed for the POC

A new dedicated **/poc** route now demonstrates direct AI work instead of requiring a full application walkthrough. It has a document upload/demo brief, extraction into editable account fields, a KYC view, a 1-5 weighted scoring view, extracted signals, and a correction prompt.

2. Next.js and TypeScript Basics for This Repo

Concept	How it appears here	What to say in presentation
App Router	Files under <code>src/app</code> define routes. Example: <code>src/app/portfolio/page.tsx</code> maps to <code>/portfolio</code> and <code>src/app/poc/page.tsx</code> maps to <code>/poc</code> .	The URL structure comes from the folder structure.
API route handlers	Files named <code>route.ts</code> under <code>src/app/api</code> implement server endpoints. Example: <code>src/app/api/ai/score/route.ts</code> .	Frontend screens call these endpoints with fetch; the server code handles DB and AI calls.
Client components	Files with 'use client' run in the browser and can use hooks/state. <code>PortfolioPage</code> and <code>/poc</code> are client components.	Interactive screens live in client components.
Server-only logic	AI providers, Prisma, parsers, and API routes stay server-side.	API keys and database access are not exposed to the browser.
TypeScript	Interfaces and types define shapes for roles, scoring, KYC, AI providers, and POC results.	Types make the data contracts explicit and reduce runtime mistakes.
Path aliases	Imports like <code>@/lib/ai</code> map to <code>src/lib/ai</code> through <code>tsconfig.json</code> .	The code avoids fragile deep relative paths.

Project scripts

- **npm run dev**: starts the local Next.js dev server.
- **npm run build**: runs the production Next build. The config skips in-build lint/type checks due local SWC issues; use TypeScript separately when possible.
- **npm run db:seed**: runs the Prisma seed script for demo data.
- **npm run vercel-build**: runs Prisma generate and Next build for deployment.

3. Repository Map

Area	Key files/folders	Responsibility
App shell	src/app/layout.tsx, src/app/page.tsx	Global layout wraps ThemeProvider, RoleProvider, RoleBar. The home page redirects to /portfolio.
Portfolio UI	src/app/portfolio/page.tsx, src/components/portfolio/PortfolioPage.tsx	Large client-side portfolio workspace, account cards, account modal, documents tab, mock account setup flow.
POC UI	src/app/poc/page.tsx	Focused demo experience for upload, extraction, correction, KYC, scoring, and signals.
API routes	src/app/api/**/route.ts	Server endpoints for accounts, scores, documents, KYC, QBR, settings, AI agents, and POC routes.
AI layer	src/lib/ai/**	Provider abstraction, OpenAI/Claude/Gemini providers, logging, and specialized agents.
Scoring	src/lib/scoring/**	KPI weight constants, deterministic KPI scoring, trigger engine, recommendation expiry.
Database	prisma/schema.prisma, prisma/migrations, prisma/seed	Prisma models, MySQL schema, seed data for demo accounts and histories.
Adapters	src/lib/adapters/**	Mock/live connector pattern for Salesforce, Jira, WorkspHERE, and Finance data.
Playbooks/RAG	src/lib/playbooks/**, src/lib/rag/README.md	File parsing and playbook extraction flow for rules and recommendations.
Shared UI	src/components/ui/**, src/styles/globals.css	Reusable Button/Input/Textarea/Card/Badge components and design tokens.

The most important mental model: pages are user-facing screens, route handlers are backend endpoints, src/lib contains reusable business logic, and Prisma owns durable data shape.

4. Runtime Flow

Current deployed walkthrough flow

- User visits /, and src/app/page.tsx redirects to /portfolio.
- src/app/layout.tsx wraps all pages with ThemeProvider, RoleProvider, and RoleBar.
- RoleBar stores the current role through RoleContext and now also links to Portfolio and POC.
- PortfolioPage is a large client component with local state for filters, modals, onboarding, documents, score rows, and account workspace tabs.
- Many portfolio cards and modal details use seeded/static V2 demo data from src/lib/v2/portfolioData.ts and constants inside PortfolioPage.

New POC flow

- User opens /poc from the top bar.
- The page posts a file or demo brief to POST /api/poc/extract.
- The extract route parses document text with src/lib/playbooks/parser.ts, then calls complete() from src/lib/ai.
- The AI returns JSON for account fields, KYC sections, signals, and 1-5 KPI dimension scores.
- The UI fills input boxes and textareas from that JSON.
- User enters a correction, such as 'ARR should be \$2.1M', and the page posts current JSON plus instruction to POST /api/poc/correct.
- The correction route asks the AI to update only supported fields, then returns recalculated scoring.

5. Data Model Summary

Model	Meaning	Presentation language
User	People using the app. Role controls what they can do.	Associate/KAM/Executive role-based workflow.
Account	Customer account core record: name, industry, region, ARR, contract dates, health.	The center of the platform.
AccountContact	Stakeholders at the customer account.	Relationship map and contact coverage.
KpiDimension	Raw KPI measurements, such as engagement, financial, or resource metrics.	Source inputs for scoring.
KamScore	Computed account health score with 8 KPI fields and AI narrative.	Explainable account health snapshot.
Signal	Risk/opportunity alert such as churn risk, contract expiry, ticket spike.	What needs attention.
Action	Human or AI-proposed next step.	The operational follow-up.
Document	Uploaded document metadata and extracted text/signals/KYC effects.	Contracts, SOWs, meeting notes, prior KYC.
KycVersion	Draft/submitted/approved KYC intelligence sections.	Structured account knowledge.
QbrSession/QbrItem	QBR/DBR/EBR planning and generated content.	Executive/account review support.
Playbook/PlaybookRule	Uploaded playbooks and extracted recommendation rules.	Codified account management policy.
Recommendation/Feed back	Recommended actions and feedback loop outcomes.	The learning loop.
AppConfig	Global score weights and notification preferences.	Configurable governance.

Database provider is MySQL through Prisma. The schema expects DATABASE_URL at runtime. Seed data builds demo accounts, scores, signals, actions, KYC versions, playbooks, and recommendations.

6. Permissions and Roles

Permissions are centralized in `src/lib/permissions/policy.ts`. API routes typically read the role from the `x-role` header using `getRoleFromRequest()`, then call `guard(role, permission)`.

Role	Core abilities
Associate	View scoped accounts, upload documents, draft/update/submit KYC, create/update actions, log touchpoints.
KAM	Full portfolio view, create/update accounts, manage contacts, upload/delete documents, approve/reject KYC, create QBR, manage opportunities, approve score changes.
Manager	Alias of KAM plus manager-level compatibility permissions.
Executive	Read-only across portfolio, scores, documents, KYC, QBR, playbooks, and exports.
Admin	Full permissions across all resources.

How to explain it

The app is designed around human approval. AI can propose information, signals, scores, and actions, but role-aware controls decide who can commit, approve, reject, dismiss, or export.

7. AI Provider Architecture

File	What it does
src/lib/ai/index.ts	Factory and complete() wrapper. Reads AI_PROVIDER, creates OpenAI/Claude/Gemini provider, logs calls.
src/lib/ai/provider.interface.ts	Shared LLMRequest, LLMResponse, and LLMProvider contracts.
src/lib/ai/providers/openai.ts	OpenAI chat completion implementation. Reads OPENAI_API_KEY and AI_MODEL/OPENAI_MODEL.
src/lib/ai/providers/claude.ts	Anthropic Claude provider. Reads ANTHROPIC_API_KEY.
src/lib/ai/providers/gemini.ts	Google Gemini provider. Reads GOOGLE_AI_API_KEY.
src/lib/ai/logger.ts	Persists or logs model calls for observability.

Important environment variables

- AI_PROVIDER: openai, claude, or gemini. Defaults to openai.
- AI_MODEL: overrides provider default model. The repo defaults to gpt-5.4-mini in multiple places.
- OPENAI_API_KEY, ANTHROPIC_API_KEY, GOOGLE_AI_API_KEY: server-side provider keys.
- DATABASE_URL: MySQL connection string for Prisma.
- ADAPTER_MODE and per-adapter mode variables: choose mock or live integration behavior.

The new POC routes use complete(), so they automatically reuse the same provider and runtime keys as the existing deployed app. No client-side key is used.

8. Scoring Engine

The existing production-style scoring route is POST /api/ai/score. It uses an internal 0-100 scale for each KPI, persists KamScore records, and then maps account health to HEALTHY, AT_RISK, or CRITICAL.

Score computation flow

- Load configurable weights from AppConfig, falling back to DEFAULT_WEIGHTS in src/lib/scoring/weights.ts.
- Load account and KPI dimensions from Prisma.
- Fetch Jira, WorkspHERE, and Finance adapter data in parallel.
- calculateKpiSubscores() computes 8 KPI dimensions and driver breakdowns.
- Questionnaire responses can blend into scores at 30% when available.
- Approved score overrides replace specific dimensions.
- Weighted score is calculated from the configured weights.
- An AI narrative is generated through complete(), with a graceful fallback if the LLM fails.
- KamScore is persisted, Account.health is updated, and trigger/orchestrator jobs run in the background.

Lead framework alignment

The lead document uses a 1-5 score scale. The POC uses that directly. Existing app scoring still stores 0-100 dimension values, but the shared weights were aligned to the lead matrix.

KPI dimension	Weight
Relationship Health	20%
Contract Health	15%
Customer Success (CSAT)	15%
Risk Score	15%
Resource Health	10%
Project Health	10%
Financial Health	10%
Whitespace Analysis	5%

9. Lead Scoring Framework Details

Overall 1-5 interpretation

Score	Status	Meaning
5	Excellent	Very Healthy
4	Good	Healthy
3	Moderate	Requires Monitoring
2	Weak	Action Required
1	Critical	Immediate Intervention Required

Portfolio bands

Overall score	Status	Portfolio classification	Recommended action
4.50 - 5.00	Excellent	Strategic Growth Account	Focus on expansion and executive alignment
3.50 - 4.49	Healthy	Stable Account	Maintain engagement and pursue opportunities
2.50 - 3.49	Watchlist	Attention Required	Address weak scoring dimensions
1.50 - 2.49	At Risk	Retention Risk	Execute corrective action plan
Below 1.50	Critical	Escalation Required	Immediate executive intervention

Criteria by KPI

KPI	Criteria
Relationship Health	Executive engagement; stakeholder coverage; relationship penetration; champion strength; engagement cadence.
Contract Health	Contract duration; notice period protection; renewability; price uplift protection; termination protection.
Customer Success (CSAT)	NPS score; customer confidence; delivery satisfaction; communication satisfaction; issue resolution.
Resource Health	Resource dependency risk; critical resource coverage; team stability; skill alignment; backup readiness.
Risk Score	Industry risk; competitive threat; vendor displacement risk; delivery risk; commercial risk.
Project Health	Delivery performance; backlog readiness; roadmap visibility; escalation status; client confidence.
Financial Health	Payment timeliness; outstanding exposure; client financial stability; revenue trend; contract vs billing alignment.
Whitespace Analysis	Service penetration; cross-sell potential; upsell potential; growth signals; expansion readiness.

10. Document Upload, Parsing, and Extraction

Existing application routes

- POST /api/documents/upload stores a file under public/uploads and creates a Document row. It requires accountId and document:create permission.
- POST /api/documents/[id]/parse reads the stored file and extracts text. PDF uses pdfjs-dist, DOCX uses mammoth, TXT reads directly.
- POST /api/ai/extract analyzes raw text with the AI provider, returns summary/key terms/obligations/signals/KYC suggestions, and persists extraction results on the Document row.

POC extraction route

- POST /api/poc/extract accepts multipart file upload or demo text.
- It parses PDF, DOCX, TXT, Markdown, XLSX, and XLS through parsePlaybookFile().
- It prompts the AI with the lead 1-5 framework and asks for a strict JSON object.
- It normalizes the result into a stable shape for the /poc UI.
- If the AI call fails, it returns a local fallback result so local demos are not blank.

11. KYC Drafting

The main KYC endpoint POST `/api/ai/kyc` delegates to `runKycDraftAgent()` in `src/lib/ai/agents/kycDraft.ts`. That agent builds a source-rich context from the account, contacts, KPIs, latest score, active signals, documents, touchpoints, Salesforce mock data, and public intelligence.

KYC stored fields

- `executiveSummary`: summary of company, relationship state, risk, opportunity, and action.
- `businessModel`: industry overview, company history, and business model.
- `keyStakeholders`: stakeholder map and contact gaps.
- `strategicGoals`: account history and goals.
- `riskFactors`: delivery, relationship, commercial, and data confidence risks.
- `expansionOpportunity`: whitespace, upsell, and Txxel team opportunity context.
- `csatHistory`: engagement history, CSAT/client feedback, projects, and relationship health.
- `competitiveLandscape`: competitors, displacement risk, and unknowns.
- `financialOverview`: ARR, contract, funding/revenue context, dependency, and renewal outlook.

The POC mirrors these same KYC fields, but keeps them stateless and editable on the `/poc` screen so the presenter can show direct extraction and correction.

12. Agents, Playbooks, and Recommendations

Area	Files	Purpose
Playbook parsing	src/lib/playbooks/parser.ts	Extract chunks from PDF/DOCX/TXT/MD/XLSX for rule extraction and source references.
Playbook extraction	src/lib/ai/agents/playbookExtractor.ts	Turn uploaded playbook content into rules with category, condition, recommendation, priority, and source metadata.
Source checker	src/lib/ai/agents/sourceChecker.ts	Validate extracted rule categories and source alignment.
Recommendation orchestrator	src/lib/ai/agents/recommendationOrchestrator.ts	Match weak scores/signals to playbook rules; fall back to AI if no rule matches.
Feedback capture	src/lib/ai/agents/feedbackCapture.ts	Capture why recommendations were actioned or dismissed.
Fallback crystallizer	src/lib/ai/agents/fallbackCrystallizer.ts	Find repeated successful AI fallback patterns and propose reusable playbook rules.
Rule quality scorer	src/lib/ai/agents/ruleQualityScorer.ts	Analyze rule performance after enough feedback signals exist.
Master orchestrator	src/lib/ai/agents/masterOrchestrator.ts	Coordinates background activity after score or signal events.

This gives you a strong presentation point: the system is not just one LLM prompt. It is a workflow where deterministic rules, source-backed playbooks, human feedback, and AI fallback all work together.

13. Integration Adapters

Adapters isolate external systems from app logic. Each adapter supports mock/live modes, so the demo can work without real integrations while the contract stays ready for production connectors.

Adapter	Folder	Signals it contributes
Salesforce	src/lib/adapters/salesforce	Opportunities, account commercial context, CRM-like information.
Jira	src/lib/adapters/jira	Open tickets, critical tickets, resolution days, sprint velocity.
WorkspHERE	src/lib/adapters/workspHERE	Meetings, sentiment, usage/utilization, resource-like context.
Finance	src/lib/adapters/finance	Invoices, overdue amount, MRR, revenue utilization, revenue history.

How to describe this

The score route does not need to know whether data came from a mock or live provider. It calls `getJiraAdapter().fetch(accountId)`, `getWorkspHEREAdapter().fetch(accountId)`, and `getFinanceAdapter().fetch(accountId)`, then computes the same scoring output.

14. POC Implementation Added

File	What was added
src/lib/poc/scoringFramework.ts	Lead framework constants, KPI definitions, 1-5 score helpers, status bands, and result types.
src/lib/poc/result.ts	Normalizes AI output into a stable POC shape and provides local fallback extraction.
src/app/api/poc/extract/route.ts	Stateless upload/text extraction route that parses files and calls the existing AI provider wrapper.
src/app/api/poc/correct/route.ts	Stateless correction route that updates current extracted JSON using a user instruction.
src/app/poc/page.tsx	Interactive POC screen with upload, demo brief, account extraction, KYC, scoring, signals, and correction panel.
src/components/layout/RoleBar.tsx	Adds Portfolio and POC navigation links in the shared top bar.
src/lib/scoring/weights.ts and src/lib/scoring/kpi.ts	Shared score weights aligned to the lead framework.
src/components/portfolio/PortfolioPage.tsx	Visible mock KPI weights aligned to the lead framework.

POC demo script

- Open the app and click POC in the top bar.
- Click Demo or upload a PDF/DOCX/TXT/XLSX account document.
- Show account fields filled automatically.
- Switch to KYC and show the 9 KYC sections generated from the source.
- Switch to Scoring and show 1-5 dimension scores, weights, evidence, risk, and action.
- Type a correction such as 'ARR should be \$2.1M and renewal date is 2026-10-15'.
- Show the fields update and scoring remain explainable.

15. Likely Presentation Questions

Question	Answer
Where are AI models used?	Through <code>src/lib/ai/complete()</code> . Existing uses include scoring narrative, KYC draft agent, document extraction, QBR generation, onboarding assistant, pulse insights, and recommendations. The POC uses the same wrapper in <code>/api/poc/extract</code> and <code>/api/poc/correct</code> .
Are API keys exposed?	No. Provider keys are read in server-side provider classes from environment variables. Client components call server API routes.
Is scoring purely AI?	No. Existing scoring is deterministic with adapter/KPI data, questionnaire blending, overrides, and configurable weights. AI adds narrative and agent recommendations. The POC uses AI for document extraction but still calculates overall score deterministically from returned 1-5 dimension scores and weights.
What happens if AI returns bad JSON?	The POC normalizer extracts and validates fields. If the AI call fails locally, a fallback result keeps the demo usable.
How does human review fit?	The main app has role-aware accept/deny/approve flows. The POC shows correction as the simplest human-in-the-loop interaction.
What is the DB?	Prisma with MySQL. <code>DATABASE_URL</code> is required for full deployed app routes.
What is mock vs live?	Adapters can run in mock or live mode. Mock lets demos work without external Salesforce/Jira/WorkspHERE/Finance integrations.
What is the proof of concept proving?	That the system can ingest a source document, extract structured account intelligence, populate KYC and account fields, score the account with the lead framework, and accept natural-language corrections.

16. Deployment Notes

- This clone currently did not include node_modules or a local .env file.
- The POC uses existing runtime AI env vars: AI_PROVIDER, AI_MODEL, OPENAI_API_KEY, ANTHROPIC_API_KEY, or GOOGLE_AI_API_KEY.
- For the full app, DATABASE_URL is needed because many existing routes use Prisma.
- For a Vercel-style deployment, make sure environment variables exist in the project settings before building.
- If deployment is GitHub-connected, pushing the branch with the /poc route should trigger deployment based on the existing deployment pipeline.
- Do not commit API keys. Keep them in deployment/runtime environment variables only.

Recommended local verification commands

- npm install
- npx prisma generate
- npx tsc --noEmit
- npm run build
- npm run dev

Final presentation close

The platform already contains a broad KAM workflow. The new POC lets you demonstrate the AI value directly: upload a source, extract intelligence, correct it conversationally, and show explainable scoring aligned to the lead's 1-5 account health framework.